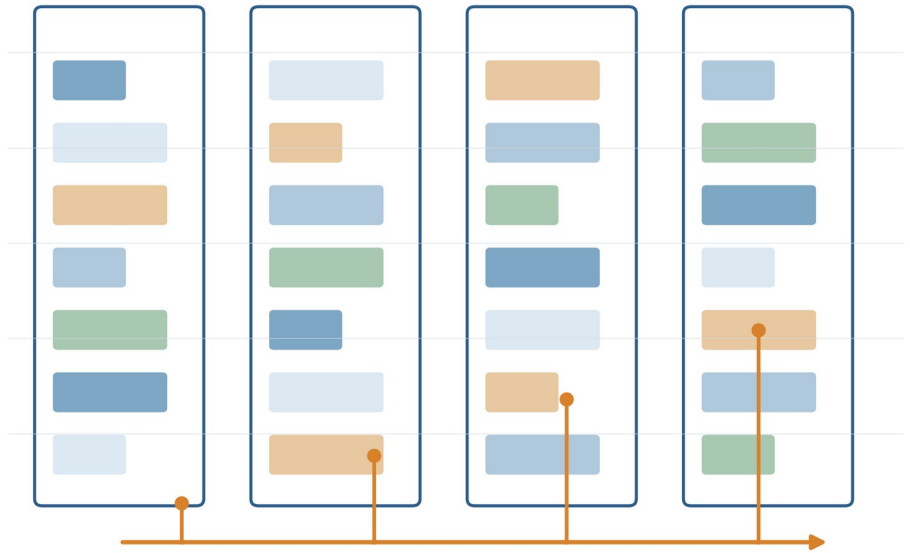
Revisiting Lifetime-Aware VM Allocation

Does better prediction make better VM placement?

Part 1 The original method
Part 2 Reproduction
Part 3 Improvement

CE7490 Course Project · 26S1 Group 5

Zhang Qi (G2510784F)
qi015@e.ntu.edu.sg
Zhang Saining (G2511061H)
saining002@e.ntu.edu.sg
Zheng Yitao (G2608664B)
YZHENG056@e.ntu.edu.sg



Based on LAVA (MLSys 2025)
Reproduction first, then improvement

VM placement uses both capacity and time

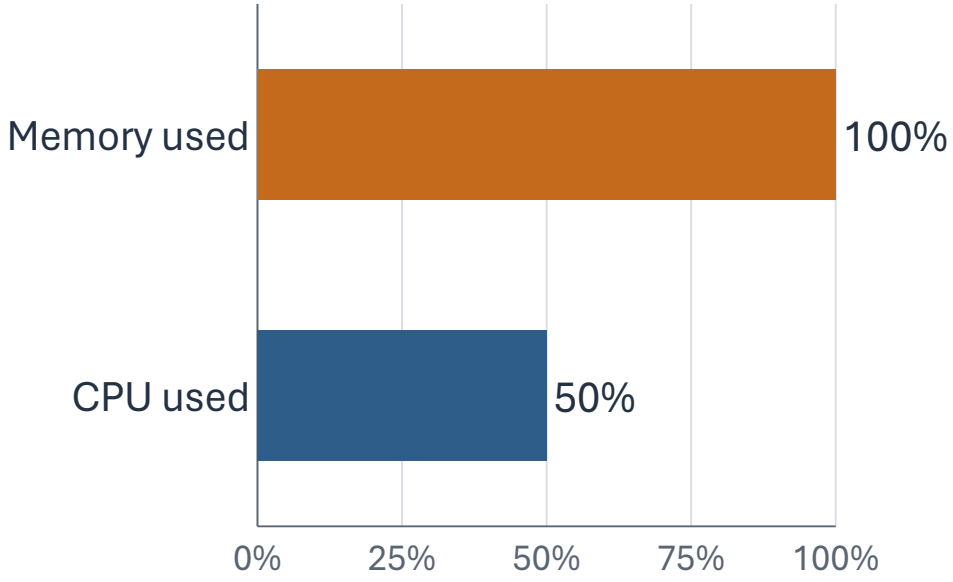
A placement must fit CPU and memory. Departures determine when a whole host becomes empty.

VM: a virtual computer

Host: a physical server running several VMs

A request fits only when BOTH resources are available.

Even one remaining VM prevents the host from becoming completely empty.



Capacity example: CPU has room, memory is full.

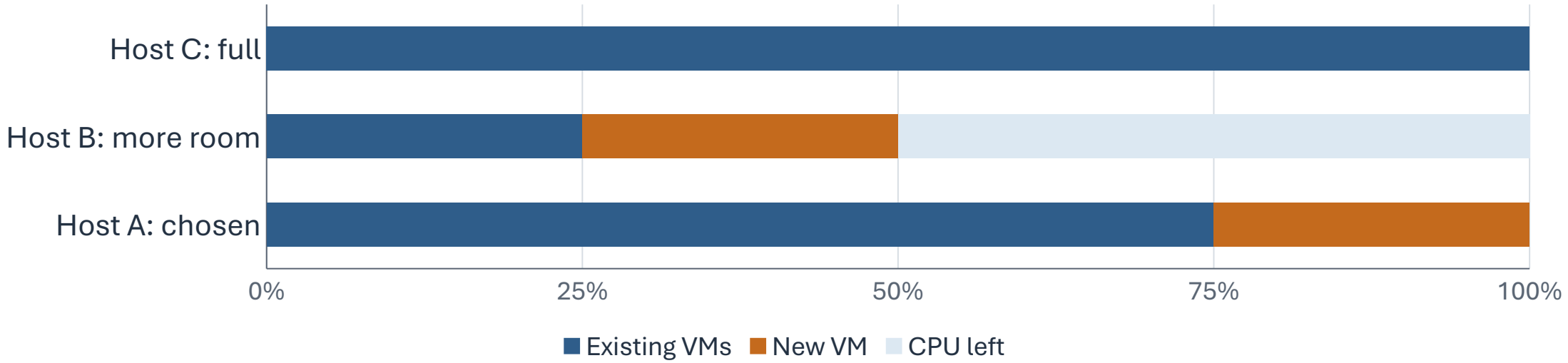
Our metric measures time with empty hosts. It does not directly measure electricity savings.

Best-Fit chooses the tightest feasible host

Best-Fit is the baseline: use the current resource space to select a feasible placement.

1 Check CPU and memory limits

2 Prefer less unused space after placement



Example VM: 25% CPU and 12.5% memory. All feasible candidates have enough memory.

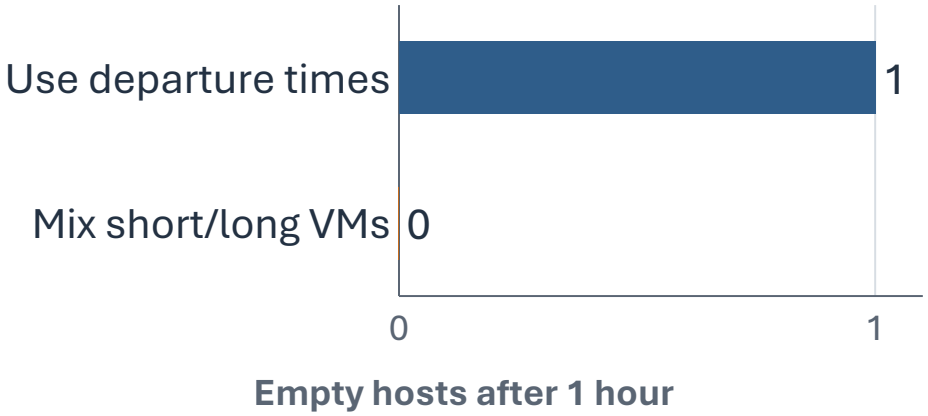
Best-Fit uses current space. LAVA additionally uses expected departure times.

LAVA adds expected departure times

The same VMs can occupy hosts for different lengths of time, depending on how they are grouped.

Four simultaneous arrivals: two run for 1 h, two for 10 h. Each host fits two VMs.

Arrangement	Host A	Host B
Mixed	1 h + 10 h	1 h + 10 h
Similar exits	1 h + 1 h	10 h + 10 h



After 1 hour: grouping similar departures frees one whole host.

This illustrates the intuition. Full LAVA also updates predictions and adapts host lifecycle states.

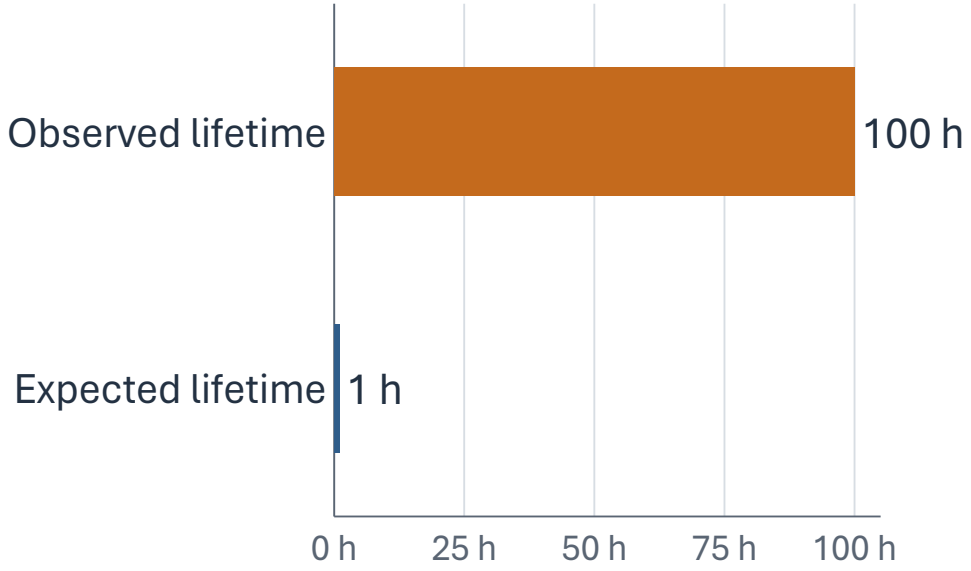
Prediction errors can reverse the benefit

An unexpectedly long VM can keep a host busy after the other VMs leave.

Illustrative prediction error

A VM looks short, so the scheduler expects its host to empty soon.

If it runs much longer, that planned departure alignment breaks.



LAVA adapts to mispredictions. We test how much the initial prediction quality still matters.

What we could reconstruct

We rebuild the published mechanisms and the available course setup, then test their behavior.

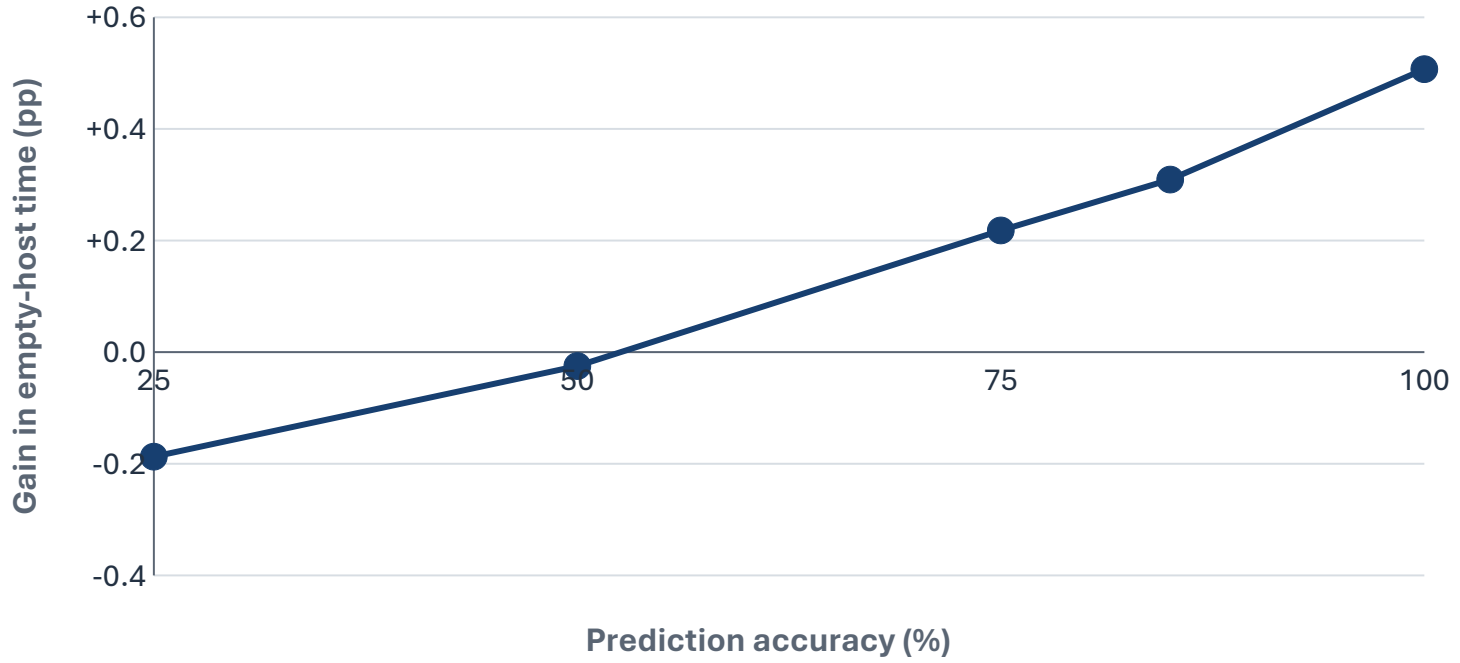
Available evidence	What we implement	What remains an assumption
LAVA paper + course artifacts	Six placement policies; host lifecycle; prediction updates	Exact original code and production traces unavailable
Course experiment settings	180 hosts; 2,500 VMs; 8 arrivals/h	Poisson arrivals; profile proportions; resource shapes
Prediction model description	Three VM profiles; controlled prediction errors	Missing mixture parameters and some update rules

This is a synthetic mechanism reconstruction, not an exact replay of the original experiments.

LAVA improves as prediction quality rises

Fresh-seed comparison: LAVA minus Best-Fit on the same 100 workloads.

Accuracy = probability of a near-correct estimate in this controlled test.



25% accuracy

−0.187 pp

95% CI [−0.240, −0.135]

100% accuracy

+0.507 pp

95% CI [+0.445, +0.569]

Poor estimates can make lifetime-aware placement worse. This motivates a way to handle unreliable predictions.

Reproducing confidence-aware routing

Alongside LAVA, we rebuild two course-project extensions that route uncertain arrivals to Best-Fit.

Confidence = probability of the lifetime class containing the estimate. Threshold: 0.3.

Mechanism	VM-level CA	Host-level CA
Decision	Choose LAVA or Best-Fit for each arrival	Choose a LAVA or Best-Fit host pool
Host sharing	Both decisions use the same hosts	Keep host modes separate until empty
Intended benefit	Avoid trusting a weak prediction	Also avoid interference between the modes

A high probability for a lifetime class need not imply that choosing LAVA will improve packing.

Reproduction experiment and implementation checks

The same workload and noisy observations are replayed for every policy in a paired comparison.

Check	Implementation	Why it matters
Workload and metric	180 hosts, CPU + memory; empty-host-time %	Same requests; no capacity oversubscription
Independent seed groups	Initial 0–19; validation 200–205; heldout 20–39	Select settings without tuning to heldout outcomes
Original reconstruction audit	1,644 runs; 16 independent tests passed	All VMs admitted; no resource-limit violations
Information boundary	Scheduler cannot read true lifetime	Only the Oracle control gets future information

Differences are percentage points (pp), with paired 95% intervals across workloads.

Which findings reproduced, and which did not?

Compare the historical course report with our heldout reconstruction: 20 paired workloads per setting.

Setting / policy vs Best-Fit	Historical course report	Our reconstruction [95% CI]
25% accuracy / LAVA	−0.005 pp	−0.137 [−0.247, −0.026] pp
85% accuracy / LAVA	+0.200 pp	+0.399 [+0.281, +0.517] pp
Perfect information / LAVA	+0.584 pp	+0.661 [+0.549, +0.774] pp
Default predictor / Host-CA	+0.241 pp	−0.045 [−0.138, +0.049] pp

The accuracy trend is similar; the reported Host-CA advantage does not reproduce here.

Historical course numbers are not the original paper's production measurements.

Diagnosing the unreproduced Host-CA benefit

We check whether the isolation mechanism failed, before interpreting its performance.

Heldout default predictor	VM-level CA	Host-level CA
Gate changes per 1,000 arrivals	318.640	318.640
Best-Fit decisions into LAVA hosts / trace	607.750	0
LAVA decisions into Best-Fit hosts / trace	319.400	0
Capacity fallback / trace	0	0

Isolation works as designed, yet its packing benefit is not confirmed.

That motivates the next step: improve the lifetime estimate itself, rather than rely only on routing.

Improvement: combine the prediction with a prior

Prior = the usual lifetime pattern for this kind of VM. Fusion combines it with a new estimate.

The prior asks: how plausible is this observation for this kind of VM?

Example	Prior mean	Noisy estimate	Fused estimate
Interactive VM 25% accuracy	1.07 h	300 h	1.52 h
Service VM 50% accuracy	85.2 h	2 h	27.0 h

An implausible observation gets less weight. A plausible observation can still update the prior.

We change the predictor while preserving the placement code.

Improvement results on 100 fresh workloads

Registered VM-CA-Bayes combines fusion with VM-level routing. Paired gains in pp, n = 100.

Accuracy	Gain over LAVA [95% CI]	Gain over Best-Fit [95% CI]
25%	+0.224 [+0.174, +0.275] Confirmed	+0.037 [-0.012, +0.086] Not confirmed
50%	+0.147 [+0.088, +0.205] Confirmed	+0.122 [+0.064, +0.180] Confirmed
75%	+0.054 [-0.001, +0.109] Not confirmed	+0.272 [+0.217, +0.327] Confirmed
85%	+0.088 [+0.037, +0.139] Confirmed	+0.398 [+0.338, +0.457] Confirmed

75% versus LAVA remains unconfirmed (Holm p = 0.11).

25% versus Best-Fit also remains unconfirmed. Intervals are unadjusted, decisions use Holm correction across 8 tests.

A useful predictor still depends on its assumptions

Robustness changes one assumption at a time, using 50 fresh workloads per setting.

Change	Observed result	Interpretation
Milder incorrect predictions	Only 1–2 of 4 intervals above zero	Smaller errors are harder to distinguish
Prior lifetimes × 0.5	At 85%: −0.116 [−0.197, −0.034] pp vs LAVA	Underestimating long lifetimes can hurt
Prior lifetimes × 2	All four gains over LAVA have positive intervals	The direction of prior error matters
Twice the arrivals and requests	Gains over LAVA at 25–75%, 85% uncertain	Load changes the size of the benefit

The failing prior case is still +0.191 pp above Best-Fit. These robustness tests are exploratory and unadjusted.

Reproduction findings and the improvement boundary

Rebuild the method, examine the gaps, then test a targeted improvement.

Reproduction

LAVA benefits from better predictions.

The historical Host-CA advantage does not reproduce in our synthetic setting.

Checks support the implemented mechanisms, not exact equivalence to missing code.

Improvement

Prior fusion improves on LAVA in 3 of 4 planned tests.

Known priors and a known error model are strong assumptions.

Next: learn priors from history and test real traces.

Code, frozen settings and per-workload results are available in the project repository.